UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	R. A. Rafa Lutfyah (2428240125)
Kelas	SI5C
Judul project / topik API	Tugas 1 RESTful API - Topik 14: Maskapai, Jadwal Penerbangan (/flights)
Tautan repositori	https://github.com/raden2428240125/tugas1-restful-2428240125
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	17 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:
API ini adalah RESTful API dari Tugas 1 untuk Topik 14, Maskapai: Jadwal Penerbangan. Entitas utamanya adalah flight (jadwal penerbangan) pada alamat /flights dengan atribut kodePenerbangan, maskapai, asal, tujuan, dan jadwalBerangkat. API dibuat dengan Express.js dalam satu berkas app.js, datanya disimpan sementara di array flights pada memori, dan tidak memakai API key. Respons GET dikirim polos, sedangkan POST, PUT, DELETE, dan error memakai struktur { status, message, data }.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/flights	Tidak	200	Ambil semua data (mendukung filter)
2	GET	/flights/:id	Tidak	200	Ambil satu data berdasarkan id
3	POST	/flights	Tidak	201	Tambah jadwal penerbangan baru
4	PUT	/flights/:id	Tidak	200	Ganti seluruh data pada id tertentu
5	DELETE	/flights/:id	Tidak	200	Hapus data (balas pesan, data: null)
6	GET	/flights?asal=CGK	Tidak	200	Filter data berdasarkan kode bandara asal
7	GET	/	Tidak	200	Info API (nama, npm, kelas, topik, endpoint)

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

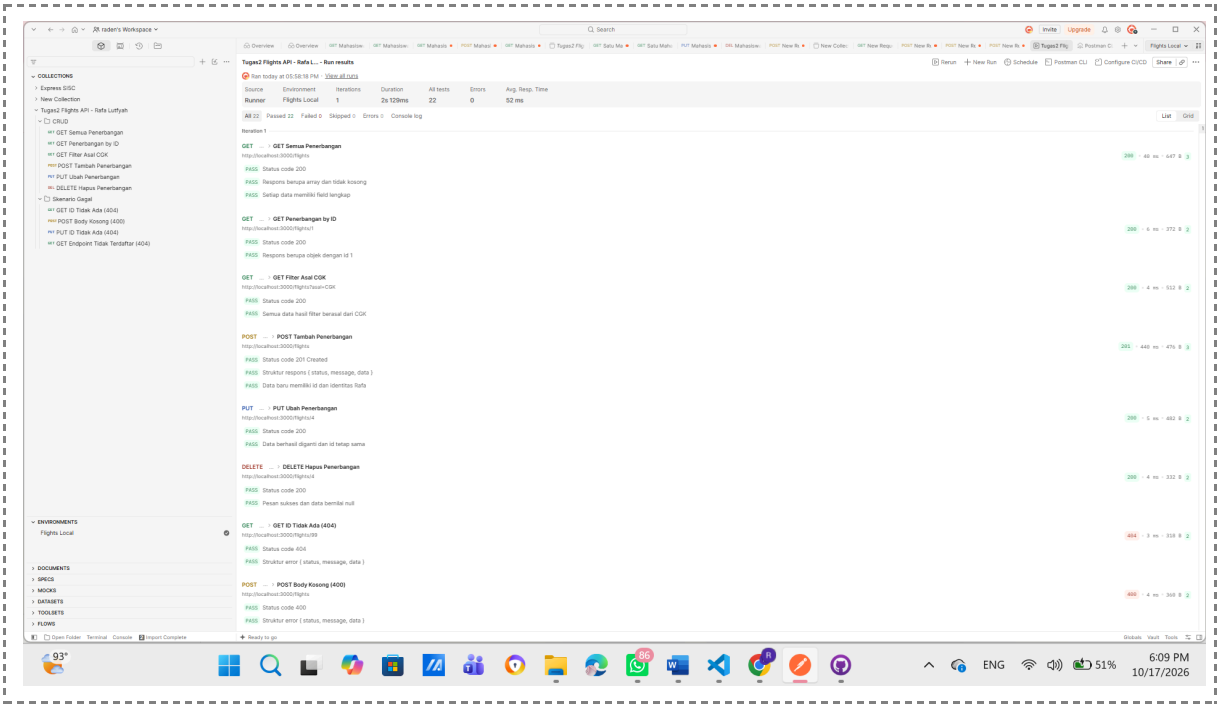
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server
2	flightId	(kosong, diisi script)	ID dari respons POST, dipakai PUT dan DELETE

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:
Saya membuat satu Collection "Tugas2 Flights API - Rafa Lutfyah" dengan dua folder. Folder CRUD berisi enam request berurutan: GET semua data, GET berdasarkan id, GET filter asal, lalu POST, PUT, dan DELETE. Urutan POST sebelum PUT dan DELETE penting karena POST menyimpan id data baru ke variabel flightId yang dipakai kedua request sesudahnya. Folder Skenario Gagal berisi empat request yang sengaja salah (GET id 99, POST body kosong, PUT id 99, dan GET endpoint tidak terdaftar) untuk membuktikan API membalas 400 atau 404 dalam bentuk JSON.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	GET Semua Penerbangan	GET	{{base_url}}/flights	CRUD
2	GET Penerbangan by ID	GET	{{base_url}}/flights/1	CRUD
3	GET Filter Asal CGK	GET	{{base_url}}/flights?asal=CGK	CRUD
4	POST Tambah Penerbangan	POST	{{base_url}}/flights	CRUD

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
5	PUT Ubah Penerbangan	PUT	{{base_url}}/flights/{{flightId}}	CRUD
6	DELETE Hapus Penerbangan	DELETE	{{base_url}}/flights/{{flightId}}	CRUD
7	GET ID Tidak Ada (404)	GET	{{base_url}}/flights/99	Skenario Gagal
8	POST Body Kosong (400)	POST	{{base_url}}/flights	Skenario Gagal
9	PUT ID Tidak Ada (404)	PUT	{{base_url}}/flights/99	Skenario Gagal
10	GET Endpoint Tidak Terdaftar (404)	GET	{{base_url}}/tidak-ada	Skenario Gagal

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	GET Semua Penerbangan	Status code 200	Respons berupa array tidak kosong; setiap data punya enam field lengkap
2	GET Penerbangan by ID	Status code 200	Respons berupa objek dengan id 1 dan kodePenerbangan XY-512
3	GET Filter Asal CGK	Status code 200	Array tidak kosong dan semua data memiliki asal CGK
4	POST Tambah Penerbangan	Status code 201 Created	Struktur { status, message, data }; data punya id angka dan identitas NPM; id disimpan ke flightId
5	PUT Ubah Penerbangan	Status code 200	status success; id sama dengan flightId; tujuan menjadi SUB dan jadwal berubah
6	DELETE Hapus Penerbangan	Status code 200	status success, pesan berhasil dihapus, data bernilai null; flightId dibersihkan
7	GET ID Tidak Ada (404)	Status code 404	status error, pesan memuat id 99 dan tidak ditemukan, data null
8	POST Body Kosong (400)	Status code 400	status error, pesan wajib diisi, data null
9	PUT ID Tidak Ada (404)	Status code 404	status error, pesan tidak ditemukan, data null
10	GET Endpoint Tidak Terdaftar (404)	Status code 404	status error, pesan Endpoint tidak ditemukan, data null

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Test script request GET: GET Semua Penerbangan
pm.test("Status code 200", function () {
    pm.response.to.have.status(200);
});
pm.test("Respons berupa array dan tidak kosong", function () {
    const json = pm.response.json();
    pm.expect(json).to.be.an("array");
    pm.expect(json.length).to.be.above(0);
});
pm.test("Setiap data memiliki field lengkap", function () {
    const fields = [
        "id", "kodePenerbangan", "maskapai",
        "asal", "tujuan", "jadwalBerangkat"
    ];
    pm.response.json().forEach(function (f) {
        pm.expect(f).to.have.all.keys(fields);
    });
});
```

Penjelasan:
Test pertama memeriksa status code 200 untuk memastikan GET /flights berhasil. Test kedua memeriksa bahwa respons berupa array dan tidak kosong, sesuai aturan Tugas 1 bahwa GET mengirim data polos, bukan objek { status, message, data }. Test ketiga mengulang setiap data dan memakai all.keys untuk memastikan tiap jadwal punya tepat enam field (id, kodePenerbangan, maskapai, asal, tujuan, jadwalBerangkat), sehingga field yang hilang atau berlebih langsung ketahuan.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Test script request POST: POST Tambah Penerbangan
pm.test("Status code 201 Created", function () {
    pm.response.to.have.status(201);
});
pm.test("Struktur respons { status, message, data }", function () {
    const json = pm.response.json();
    pm.expect(json.status).to.eql("success");
    pm.expect(json.message).to.include("berhasil ditambahkan");
    pm.expect(json.data).to.be.an("object");
});
pm.test("Data baru memiliki id dan identitas Rafa", function () {
    const data = pm.response.json().data;
    pm.expect(data.id).to.be.a("number");
    pm.expect(data.maskapai).to.include("2428240125");
});
// simpan id data baru untuk request PUT dan DELETE (hanya bila POST berhasil)
if (pm.response.code === 201) {
    pm.environment.set("flightId", pm.response.json().data.id);
}
```

Penjelasan:

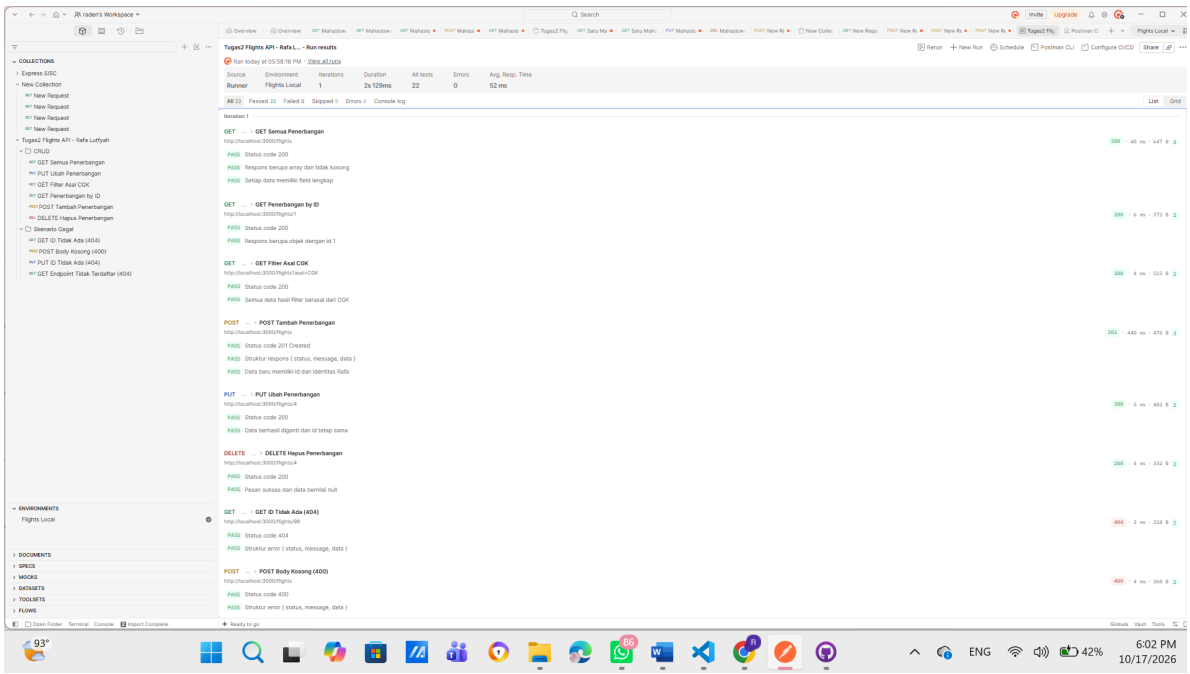
Pada request POST Tambah Penerbangan, id dari respons (data.id) saya simpan ke variabel Environment flightId lewat pm.environment.set, tetapi hanya bila status 201 agar script tidak error ketika POST gagal. Request PUT Ubah Penerbangan dan DELETE Hapus Penerbangan memakai {{flightId}} pada alamat /flights/{{flightId}}. Bila urutan ditukar sehingga PUT atau DELETE berjalan lebih dulu, flightId masih kosong, alamatnya menjadi /flights/, dan server membalas 404 Endpoint tidak ditemukan sehingga test status code gagal.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:
Sebelum menjalankan Runner, saya menyalakan ulang server dengan npm run dev agar array flights kembali ke tiga data awal, memilih Environment Flights Local, lalu menjalankan seluruh Collection. Runner mengeksekusi 10 request dengan 22 test, dan seluruhnya lulus tanpa test gagal.

4.2 Rekap angka hasil

Nama Collection	Tugas2 Flights API - Rafa Lutfyah
Environment yang dipakai	Flights Local
Jumlah request dijalankan	10
Jumlah test lulus	22
Jumlah test gagal	0
Tanggal dan jam pengujian	17 Oktober 2026 pukul 18.03

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	POST, PUT, DELETE, dan 3 request skenario gagal	expected response to have status code 201 but got 401 (10 dari 22 test gagal)	Server yang menyala adalah project kuis (si5c_kuis1_rafa) yang memakai API key, bukan server Tugas 1	Mematikan server kuis, membuka folder tugas1-restful-2428240125, menjalankan npm run dev, lalu Rerun sampai 22 test lulus

Bab 5 Newman di Terminal (Pengayaan)

Bagian ini opsional
Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Tidak dikerjakan
Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

[Tempel screenshot di sini]
Gambar 3. Hasil menjalankan Collection dengan Newman

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/raden2428240125/tugas1-restful-2428240125
Lokasi berkas Collection (.json)	postman/Flights_API.postman_collection.json
Lokasi berkas Environment (.json)	postman/Flights_Local.postman_environment.json
Berkas Environment tidak memuat kunci rahasia sungguhan	[x] Ya [ ] Tidak
Repositori dapat dibuka dosen	[x] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:
Menurut saya assertion yang paling berguna adalah pemeriksaan struktur respons, seperti all.keys pada GET Semua Penerbangan dan pengecekan status, message, serta data null pada skenario gagal. Status code saja tidak cukup, karena API bisa membalas 200 tetapi bentuk datanya salah, misalnya field jadwalBerangkat hilang atau error tidak dibungkus { status, message, data }. Assertion struktur membuat perubahan seperti itu langsung terdeteksi.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:
Pada pengujian manual saya harus mengirim satu per satu dan membaca respons dengan mata, sedangkan Collection Runner menjalankan 10 request sekaligus dan langsung menunjukkan jumlah test lulus dan gagal. Bagian tersulit adalah membuat PUT dan DELETE memakai id dari POST, yang saya atasi dengan menyimpan id ke variabel flightId dan memakainya pada alamat. Saya juga belajar bahwa server harus dinyalakan ulang sebelum Runner supaya data awal kembali dan hasilnya konsisten.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	
2	Setiap endpoint API saya ada di Collection	
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	
4	Setiap request punya assertion status code dan struktur JSON	
5	Nama setiap test berupa kalimat yang menyatakan harapan	
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	
7	Server dinyalakan ulang sebelum Runner dijalankan	
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	
9	Berkas Environment tidak memuat kunci rahasia sungguhan	
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen